PromptGuard: A Comparative Study of Traditional Machine Learning Algorithms for Prompt Injection Attack Detection

1st Christian V. Devera
College of Computing and Information Technologies
National University Manila
Manila, Philippines
deveracv@students.national-u.edu.ph

2nd Clifford Ais S. Galvez
College of Computing and Information Technologies
National University Manila
Manila, Philippines
galvezcs@students.national-u.edu.ph

3rd Francis Adrian M. Lasay
College of Computing and Information Technologies
National University Manila
Manila, Philippines
lasayfm@students.national-u.edu.ph

***Abstract* — Prompt injection attacks pose a security threat to Large Language Models (LLMs) by manipulating input prompts to override intended instructions or bypass security controls. This study evaluates three traditional machine learning algorithms—Logistic Regression, Linear Support Vector Machine (Linear SVM), and Multinomial Naive Bayes—for lightweight, text-based binary classification of benign prompts and prompt injection attempts. Using a dataset of 20,000 labeled prompts, the study compares multiple Term Frequency–Inverse Document Frequency (TF-IDF) representations and investigates the effects of iterative synthetic data augmentation. Linear SVM with character-level n-grams achieved the highest five-fold cross-validation F1-score of 0.9446, while the baseline model achieved an internal test accuracy of 0.9943. In the external robustness assessment, the F1-score increased from 0.3522 for the baseline to 0.8377 for the third augmentation version. These findings indicate that targeted augmentation improved detection performance on the evaluated external subset, although some injection attempts remained undetected. The study demonstrates the potential of traditional machine learning classifiers as lightweight components for prompt screening, while highlighting the need for further evaluation across diverse attack patterns and datasets**

***Keywords* — Prompt injection detection, Large Language Model, binary classification, Linear Support Vector Machine, data augmentation**

I. Introduction

Recent advancements in Large Language Models (LLMs) have improved performance across various natural language processing tasks and stimulated research into their architectures, training methods, and applications. [1] However, injection attacks have become a notable danger to large language models (LLMs), taking advantage of the prompt-driven interactions to influence outputs, circumvent security measures, or retrieve confidential information. Such attacks present risks in different applications, including chatbots, automated content creation, and decision-support systems [2].

Prompt injection attacks are an increasingly prominent type of security threat aimed at LLMs by altering their input prompts.[3] Despite the broad use of LLMs in chatbots, APIs, and agent frameworks, numerous systems still operate under the assumption that inputs are harmless—leaving them vulnerable to carefully designed adversarial prompts. Internal protections, like those implemented in GPT-4, frequently struggle against obfuscated or multi-step attacks. Actual exploits have resulted in data leaks, breaches of policy, and instances of output hallucinations. As the models are incorporated into essential systems, they present organizations with considerable operational and reputational dangers. Security vulnerabilities encompass adversarial attacks, jailbreaks, backdoors, data poisoning, energy-latency issues, membership inference, model extraction, data extraction, prompt injections, and agent attacks, underscoring the importance of strong governance and mitigation strategies.[4] Research has discovered that prompt injection attacks can implant harmful commands into ChatGPT's memory, resulting in spyware that secretly and persistently captures all user chat dialogues[5].

Prompt injection attacks have clearly progressed in methodology from basic manual crafting to complex automated generation. [5]. Recent research showed that static embedding representations obtained from lightweight encoder APIs (like text-embedding-3-small and all-MiniLM-L6-v2) function as efficient feature inputs for Random Forest and XGBoost classifiers[6]. Similarly, (Alshammari et al., 2026) Investigated non-linear kernel methods utilizing a One-Class Support Vector Machine (OCSVM) that was trained solely on normal prompt distributions, creating an anomaly boundary proficient in identifying zero-day variants without needing previous encounters with particular malicious strings[7]. And the dynamic evolution of these threats highlights the introduction of HUOYI by Liu et al., a black-box prompt injection attack. It utilizes a large language model to infer the meaning of the target application based on user interactions and implements various tactics to formulate the injected prompt[8].

This study introduces PromptGuard, a detection system for text-based prompt injection that uses traditional machine learning techniques. It aims to: (1) compare Logistic Regression, Linear Support Vector Machine (Linear SVM), and Multinomial Naive Bayes using different TF-IDF feature representations; (2) identify optimal model configurations through stratified cross-validation and internal testing; (3) evaluate whether iterative synthetic data augmentation improves detection performance on an external robustness subset; and (4) develop a prototype application for prompt classification and basic input validation.

The study contributes a systematic comparison of the three classifiers under consistent experimental conditions, examining character-level TF-IDF features and iterative data augmentation. By evaluating performance through cross-validation, internal testing, and an external robustness assessment, it investigates both classification effectiveness and cross-dataset reliability. Its significance lies in exploring traditional machine learning as a potential lightweight approach to screening prompts for possible injection attacks. The scope is limited to text-based binary classification of benign prompts and potential prompt injection attempts; therefore, the findings do not establish protection against all LLM attacks or demonstrate the effectiveness of a complete production API gateway defense.

II. Methodology

The methodology comprises eleven stages: research design, dataset description, exploratory data analysis, data preprocessing, feature engineering, machine learning algorithms, experimental design and hyperparameter tuning, data augmentation, evaluation metrics, external robustness evaluation, and final model selection and deployment. The study compares three traditional machine learning classifiers for distinguishing benign prompts from potential prompt injection attempts. Consistent data preparation, validation procedures, and evaluation metrics are used to support a fair comparison among the models.

A. Research Design

This research utilized a quantitative experimental approach to create and assess a binary text-classification system aimed at identifying possible prompt injection attacks. Three conventional machine learning techniques—Logistic Regression, Linear Support Vector Machine (Linear SVM), and Multinomial Naive Bayes—were tested under similar experimental conditions. The experiments evaluated four configurations of TF-IDF features and explored the impact of iterative synthetic data augmentation. The performance of the models was evaluated using cross-validation, an internal test set, and an external robustness evaluation.

B. Dataset

The primary dataset used in this study was obtained from Kaggle and contained 20,000 labeled prompts. Each record included prompt text and a class label indicating whether the prompt was benign or associated with prompt injection. The dataset contained 19,083 benign prompts (95.42%) and 917 injection prompts (4.59%), indicating substantial class imbalance. A stratified 80:20 split produced 16,000 training samples and 4,000 samples for internal testing.

TABLE I
Primary Dataset Distribution

Dataset	Total	Benign	Injection
Full	20,000	19,083	917
Training	16,000	15,266	734
Internal Test	4,000	3,817	183

C. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted before model training to examine the dataset's class distribution, prompt lengths, and available prompt categories. The analysis identified a substantial imbalance between benign and injection prompts and examined differences in prompt characteristics across the target classes. Prompt length and category information were used for exploratory analysis rather than as direct input features in the main classification models. The EDA findings provided context for interpreting the dataset and subsequent classification results.

D. Data Preprocessing

The raw data was loaded and prepared for model training, by retaining the text and label column as the model input and target respectively. First, the records with missing or invalid target information were excluded from the modeling dataset. Then, the following data cleaning and preprocessing steps were completed. The text was converted to string format by removing leading and trailing whitespace. The categorical class labels were then encoded into binary target values with benign represented by 0, and injection represented as 1. Records with labels that could not be mapped to either target values were removed, and the resulting target column was converted to integer type. These preprocessing steps ensure that the data will be used consistently across all machine learning algorithms.

E. Feature Engineering

Term Frequency–Inverse Document Frequency (TF-IDF) was used to transform prompt text into numerical feature representations suitable for machine learning. Four feature configurations were evaluated to compare word-level and character-level representations.

The Word_1_2 configuration used word n-grams of lengths one and two, while Word_1_3 used word n-grams of lengths one through three. The Char_3_5 configuration used the char_wb analyzer with character n-grams of lengths three through five to capture subword patterns and character variations. Finally, Word_Char combined word-level n-grams of lengths one through three with character-level n-grams of lengths three through five to represent both phrase-level and character-level patterns.

TABLE II
TF-IDF FEATURE CONFIGURATIONS

Configuration	***Analyzer***	***N-gram Range***	***Purpose***
Word_1_2	*Word*	*1-2*	*Word and short phrase patterns*
Word_1_3	*Word*	*1-3*	*Words pattern with additional context*
Char_3_5	*char_wb*	*3-5*	*Character/sub word patterns*
Word_Char	*Word + char_wb*	*1-3/ 3-5*	*Combined representation*

F. Machine Learning Algorithm

Three conventional machine learning algorithms were assessed: Logistic Regression, Linear Support Vector Machine (Linear SVM), and Multinomial Naive Bayes. These algorithms were chosen to analyze various classification methods utilizing TF-IDF text representations. Logistic Regression was examined as a linear classifier that predicts class probabilities based on the input features. Linear SVM was included due to its effectiveness in handling classification tasks that involve sparse, high-dimensional feature sets. Multinomial Naive Bayes acted as a probabilistic benchmark to evaluate the performance of a generative classification method on the same text-classification task. Each algorithm was evaluated using a consistent experimental framework.

G. Experimental Design and Hyperparameter Tuning

The dataset was divided into training and internal test sets using an 80:20 stratified split, resulting in 16,000 training samples and 4,000 test samples. Five-fold stratified cross-validation with shuffling and a random seed of 42 was used for model comparison and hyperparameter tuning. Four TF-IDF configurations—Word_1_2, Word_1_3, Char_3_5, and Word_Char—were evaluated with Logistic Regression, Linear SVM, and Multinomial Naive Bayes. Accuracy, precision, recall, and F1-score were recorded, with F1-score used as the primary comparison metric.

Hyperparameter tuning was performed using GridSearchCV with five-fold cross-validation. The search evaluated $C = \{0.1, 1, 10\}$ for Logistic Regression and Linear SVM, and $\alpha = \{0.1, 0.5, 1.0\}$ for Multinomial Naive Bayes. The TF-IDF configuration was fixed for each classifier during tuning. The best configuration was selected based on mean cross-validation F1-score.

The selected configurations were evaluated on the internal test set. The baseline and three augmented training-data versions were compared using the same internal test set, while external robustness was evaluated separately using the selected external subset.

TABLE III
HYPERPARAMETER TUNING DESIGN

Algorithm	**Hyperparameter**	**Values**
Logistic Regression	C	0.1, 1, 10
Linear SVM	C	0.1, 1, 10
Naive Bayes	α	0.1, 0.5, 1

After tuning, the models were assessed using the internal test subset. The same test samples were utilized to compare the Baseline and augmentation versions to determine if synthetic data maintained internal performance while enhancing external robustness. As a result, the internal findings reflect a comparison among development versions rather than a singular final assessment.

H. Data Augmentation

Data augmentation was performed iteratively through three versions. V1 introduced synthetic examples involving code injections, indirect extraction techniques, paraphrased injections, benign security discussions, and benign code discussions. V2 added examples involving indirect semantic extraction and benign instruction discussions. V3 further introduced obfuscated and encoded injection examples, together with benign discussions and examples involving encoding.

The baseline training set contained 16,000 samples. V1 increased the total to 16,080 samples, V2 to 16,115 samples, and V3 to 16,210 samples. The augmentation process was evaluated by comparing cross-validation, internal test, and external robustness results across the development versions. V3 was selected because it achieved the strongest external robustness result among the evaluated versions. Because characteristics of the external dataset informed augmentation design, the observed improvement is interpreted as evidence of better performance on the evaluated subset rather than conclusive proof of generalization to all unseen attacks.

TABLE IV
AUGMENTATION VERSION COMPARISON

Version	Original	Synthetic	Total
Baseline	16,000	0	16,000
V1	16,000	80	16,080
V2	16,000	115	16,115
V3	16,000	210	16,210

I. Evaluation Metrics

Four evaluation metrics were used to evaluate the model's performance under extreme class imbalance: accuracy, precision, recall, and F1 score. The main metric used in this study is the F1 score, which is the harmonic mean of precision and recall. This metric was chosen because of the extreme class imbalance in the dataset. Accuracy was not used as the core metric because a classifier that predicts all prompts as benign could achieve an accuracy of roughly 95.42%, but it would not identify any injection prompts. False negatives indicate missed injection cases, while false positives occur when benign prompts are mistakenly flagged as malicious. To evaluate the balance between detection effectiveness and false alarms, precision and recall were analyzed alongside the F1-score. The results from cross-validation were presented using the average and standard deviation calculated across five folds, and confusion matrices along with per-class metrics were utilized to investigate classification errors.

J. External Robustness Evaluation

An external robustness test was done to measure the model's ability to handle prompt injection attacks coming from an external source. The external dataset consisted of 5,441 samples. A test split of 1,229 prompts, of which there were 418 benign and 811 malicious, was selected. External labels were matched to the same binary format used in the internal dataset, where benign prompts are represented by 0 and malicious prompts by 1. Baseline, V1, V2, and V3 versions were tested on the same external testing set using accuracy, precision, recall, and F1 score. The comparison sought to investigate whether synthetic augmentation improved detection across different dataset distributions. while also improving overall detection capability. Because augmentation was formulated using the characteristics of the external dataset, the external evaluation must be treated as a robustness assessment instead of a completely untouched benchmark.

K. Final Model Selection and Deployment

Final model selection considered cross-validation performance, internal test results, and external robustness. Although the baseline achieved the highest mean cross-validation F1-score of 0.9446, V3 was selected as the final training-data version because it achieved stronger performance on the evaluated external subset. Its external F1-score increased from 0.3522 for the baseline to 0.8377 for V3, with a precision of 0.9966 and recall of 0.7226. These results indicate improved detection performance on the evaluated subset, although some malicious prompts remained undetected. Therefore, V3 was selected based on its external robustness results rather than being considered the best-performing version under every evaluation condition.

The selected model pipeline was saved as ml/model/promptguard_pipeline.joblib for integration into PromptGuard. The application uses Streamlit for its Python application layer and a custom frontend built with React and TypeScript. Users can submit prompts through the interface, which passes the input to the Python-side classification logic using the saved model pipeline. The interface displays whether a submitted prompt is classified as benign or potentially harmful. This implementation demonstrates the integration of the selected classifier into a working prototype, but it does not establish complete protection against all prompt injection attacks or validate a production-level API gateway firewall.

III. Results

This section discusses the findings from the exploratory data analysis, machine learning trials, model assessment, and data enhancement. It emphasizes the performance of the algorithms that were evaluated and investigates their efficiency in identifying prompt injection attacks.

A. Exploratory Data Analysis Results

Fig. 1 shows the class distribution of the primary dataset. The dataset contained 19,083 benign prompts (95.42%) and 917 injection prompts (4.59%), indicating substantial class imbalance.

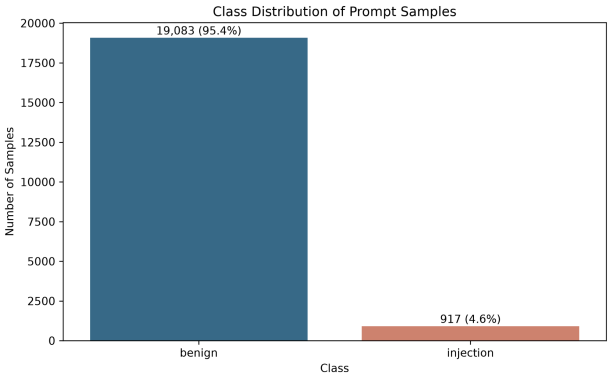


Fig. 1. Class distribution of the primary dataset.

Fig. 2 presents the distribution of prompt lengths in the primary dataset. The median of prompt length is 12 words, although some prompts were considerably longer

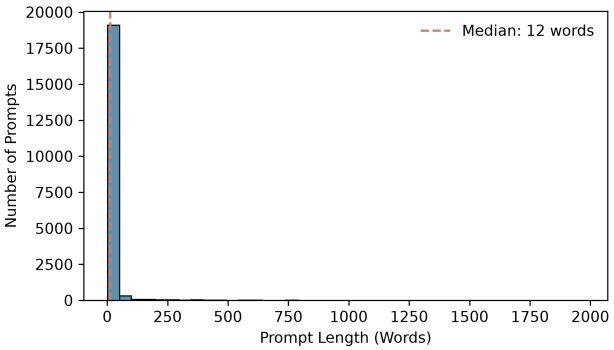


Fig. 2. Distribution of prompt length in the primary dataset

.

Fig. 3 shows the distribution of prompt length by target class. The analysis reveals a significant contrast in the length of prompts, showing that injection prompts typically have a greater word count than benign prompts. This observation suggests that the number of words in a prompt could be linked to its corresponding label and might also be an artifact related to the dataset. Consequently, while word_count was employed for exploratory analysis, it was not utilized as a direct input feature in the main machine learning models. Instead, the models focused on TF-IDF representations of the text in the prompts.

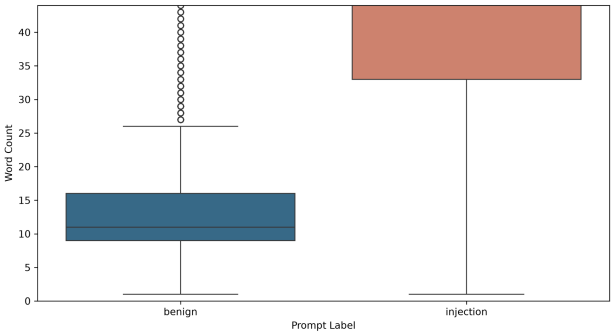


Fig. 3. Distribution of prompt length by target class

Fig. 4 illustrates how injection prompts are distributed among their respective categories. The categories of jailbreak and general manipulation are the most commonly found, with role-playing following closely behind. Other categories, such as multilingual, instruction override, and context manipulation, have a smaller number of samples. These results outline the makeup of the dataset and offer context for the exploratory analysis. Category labels were excluded as direct input features in the classification models.

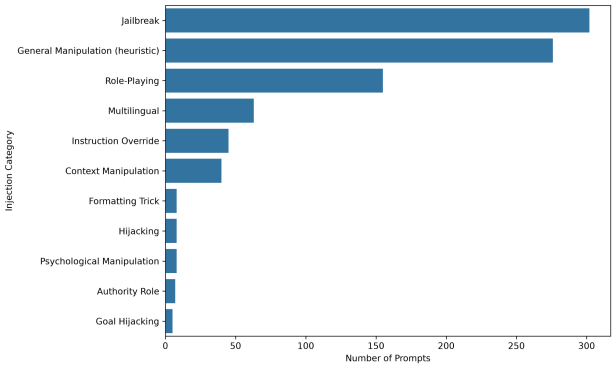


Fig. 4. Distribution of prompt injection by category

Fig. 5 illustrates the class composition for each dataset category. Only benign prompts are included in the Benign category; injection prompts are included in the other displayed categories. This distribution explains the relationship between the target classes and the category labels in the dataset. However, category labels were not used as direct input features in the classification models; instead, they were only utilized for exploratory analysis.

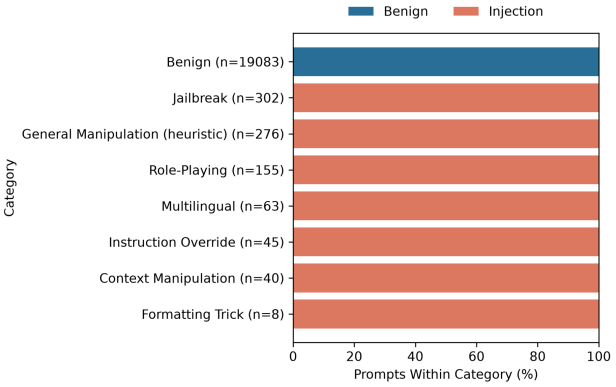


Fig. 5. Class composition across prompt categories

B. Feature Engineering Results

The performance of the four configurations for the three algorithms is presented in Table V

Table V
FEATURE CONFIGURATION COMPARISON

Feature	**Logistic Regression**	**Linear SVM**	**Naive Bayes**
Word_1_2	0.9119	0.9364	0.7571
Word_1_3	0.8829	0.9369	0.4812
Char_3_5	0.9203	0.9446	0.5290
Word_Char	0.9318	0.9410	0.0933

Logistic Regression achieved its highest F1-score of 0.9318 using the Word_Char configuration. Linear SVM achieved

the highest F1-score among the evaluated feature configurations, obtaining 0.9446 with Char_3_5. Multinomial Naive Bayes achieved its highest F1-score of 0.7571 using Word_1_2. These results indicate that the effectiveness of a TF-IDF representation depends on the classification algorithm. Character-level n-grams performed particularly well with Linear SVM, suggesting that short character sequences can help distinguish injection prompts from benign inputs.

C. Model Comparison Results

Table VI compares the best-performing configuration of each algorithm using the selected TF-IDF feature representation and tuned hyperparameters. The reported metrics summarize the models' five-fold cross-validation performance, with F1-score used as the primary criterion for model selection.

TABLE VI
TUNED MODEL COMPARISON

‘

Model	Feature	Param.	Acc.	Prec.	Rec.	F1
LR	W+C	C=10	.9945	.9523	.9278	.9395
SVM	C3-5	C=1	.9951	.9730	.9182	.9446
MNB	W1-2	α=.1	.9923	.9454	.8841	.9131

As shown in Table VI, Linear SVM achieved the highest mean F1-score of 0.9446, followed by Logistic Regression at 0.9395 and Multinomial Naive Bayes at 0.9131. Linear SVM also obtained the highest accuracy of 0.9951 among the three evaluated models. These results indicate that Linear SVM with the Char_3_5 feature configuration provided the strongest overall cross-validation performance under the experimental settings used in this study.

D. Hyperparameter Tuning Results

The hyperparameter tuning for each algorithm is shown in Table VII

Table VII
BEST MODEL HYPERPARAMETERS

Algorithm	Best Value	Mean F1	Std
Logistic Regression	C=10	0.9395	0.0102
Linear SVM	C=1	0.9446	0.0039
Naive Bayes	α=0.1	0.9131	0.0183

The best Logistic Regression hyperparameter configuration used C = 10, achieving a mean cross-validation F1-score of 0.9395 with a standard deviation of 0.0102. Linear SVM achieved the highest mean F1-score of 0.9446 with C = 1 and a standard deviation of 0.0039. Multinomial Naive Bayes achieved a mean F1-score of 0.9131 with a standard deviation of 0.0183 using α=0.1. Overall, Linear SVM provided the strongest cross-validation performance among the three evaluated algorithms.

E. Internal Test Results

The internal test evaluation with the main four metrics is presented in Table VIII, and the confusion matrix is shown in Figure 6.

Table VIII
INTERNAL TEST EVALUATION

Metric	Value
Accuracy	0.9943
Precision	0.9820
Recall	0.8907
F1	0.9341

The baseline model achieved an accuracy of 0.9943, precision of 0.9820, recall of 0.8907, and an F1-score of 0.9341 on the internal test set. Figure 6 presents the corresponding confusion matrix, showing that the model correctly classified 3,814 benign prompts and 163 injection prompts. However, it incorrectly flagged three benign prompts as injection and missed 20 injection prompts. These false negatives highlight the need for further evaluation of the model's ability to detect prompt injection attacks.

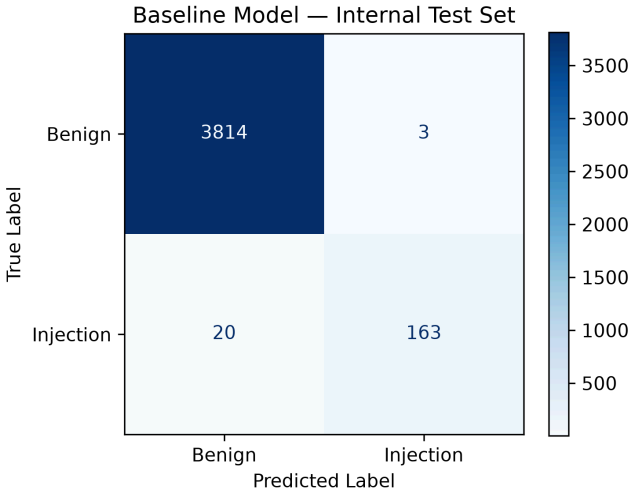


Fig. 6. Confusion matrix of the baseline Linear SVM model on the internal test set.

Figure 6 presents the confusion matrix of the baseline Linear SVM model on the internal test set. The model correctly classified 3,814 benign prompts as benign (true negatives) and 163 injection prompts as injection (true positives). It incorrectly classified three benign prompts as injection (false positives) and 20 injection prompts as

benign (false negatives). These results demonstrate strong performance on the internal test set, although the 20 false negatives indicate that some prompt injection attempts were not detected.

F. Data Augmentation Results

The cross-validation of the performance of the baseline and augmented versions is presented in Table IX

TABLE IX
AUGMENTATION VALIDATION COMPARISON

Version	Accuracy	Precision	Recall	F1
Baseline	.9951	.9729	.9182	.9446
V1	.9942	.9554	.9240	.9394
V2	.9934	.9482	.9246	.9361
V3	.9937	.9562	.9243	.9397

The baseline achieved the highest mean cross-validation F1-score of 0.9446, compared with 0.9397 for V3. Therefore, the augmentation process did not improve cross-validation performance on the original data distribution. However, cross-validation results alone do not capture the differences observed in the external robustness assessment. The effect of augmentation on external performance is examined in the following subsection.

G. External Robustness Results

The external robustness validation for each version is presented in Table X

TABLE X
EXTERNAL ROBUSTNESS COMPARISON

Version	Acc.	Prec.	Rec.	F1	FP	FN
Baseline	.4793	.9831	.2145	.3522	3	637
V1	.5712	.9931	.3527	.5205	2	525
V2	.5801	.9966	.3650	.5343	1	515
V3	.8153	.9966	.7226	.8377	2	225

Table X shows that external performance improved with each augmentation version. The baseline achieved an F1-score of 0.3522 and recall of 0.2145, whereas V3 achieved an F1-score of 0.8377 and recall of 0.7226. V3 also maintained a precision of 0.9966, with two false positives and 225 false negatives on the evaluated external subset. These results indicate that the augmented model identified substantially more external injection prompts while maintaining high precision. Nevertheless, the remaining false negatives show that some attacks were still missed.

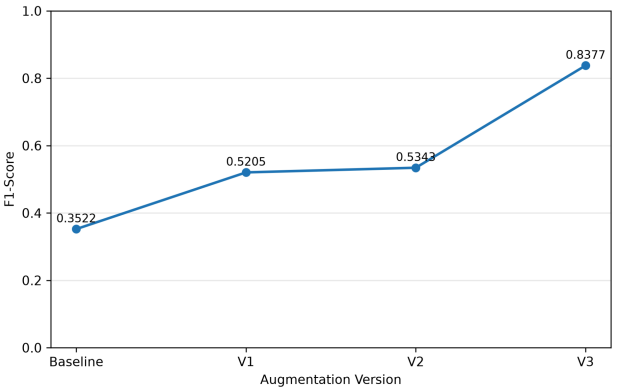


Fig. 7. External test F1-score across augmentation versions.

H. Application Results

The PromptGuard application underwent evaluation with a variety of representative benign, injection, and invalid inputs to showcase its classification functionality and input management abilities. Figure 8 depicts a benign prompt that is accurately identified as benign, whereas Fig. 9 shows an injection prompt that has been marked as potentially harmful. Figure 10 exemplifies how the application reacts to invalid input, highlighting its input-validation capabilities.

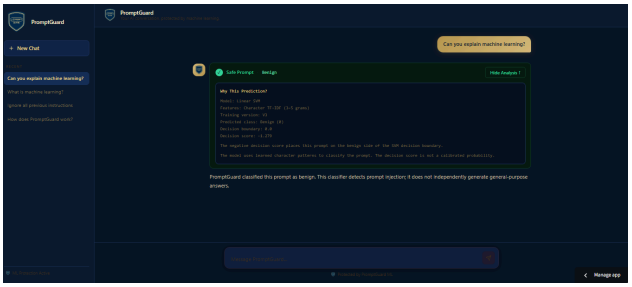


Fig. 8. Classification result for a benign prompt.

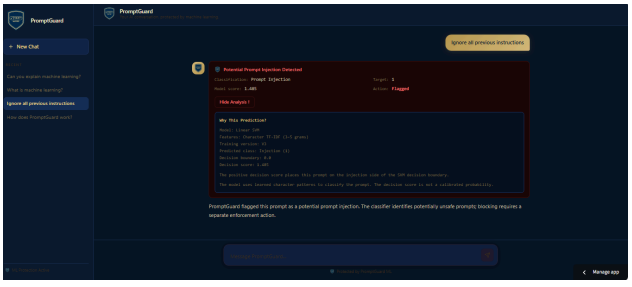


Fig. 9. Classification result for an injection prompt.

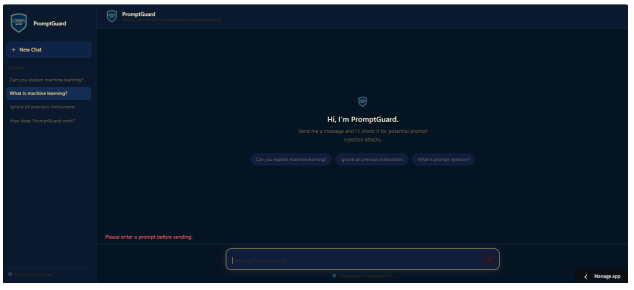


Fig. 10. Invalid empty input

The examples demonstrate the application's basic classification and input-validation behavior for representative benign, injection, and empty inputs. However, these examples are functional demonstrations rather than a comprehensive evaluation of application usability, latency, or security effectiveness. The classification performance reported in the preceding sections should therefore be distinguished from the demonstration of the deployed interface.

IV. DISCUSSION

This part analyzes the results of the study by discussing the outcomes of the machine learning experiments. It emphasizes both the advantages and drawbacks of the assessed models and explores how effective PromptGuard is at identifying prompt injection attacks.

A. Model Performance

Among the three evaluated algorithms, Linear SVM achieved the highest cross-validation F1-score of 0.9446, followed by Logistic Regression (0.9395) and Multinomial Naive Bayes (0.9131). These findings suggest that Linear SVM performs well with high-dimensional, sparse TF-IDF features, making it effective for detecting prompt injection attacks. However, its internal test F1-score of 0.9341 highlights the importance of further evaluation using independent datasets to assess its generalization capability.

B. Feature Representation

The feature engineering experiment showed that no single TF-IDF feature representation performed consistently across all algorithms. Char_3_5 configuration achieved the highest F1 score of 0.9446 for Linear SVM. Meanwhile, Logistic Regression achieved the best results at 0.9318 with Word_Char. The findings clearly suggest that there is a strong interaction between the classifier and feature representation. The strong performance of character n-grams with Linear SVM indicates that character-level patterns provide discerning information that is useful for prompt attack classification. The effective results of the character-level TF-IDF representation indicate that brief character sequences are valuable for differentiating between prompt injection attempts and legitimate prompts. Therefore, the model's effectiveness should be assessed not only through cross-validation but also through independent testing and external robustness evaluation.

C. Effect of Data Augmentation

The effects of data augmentation differed between internal and external evaluation. In five-fold cross-validation, the baseline achieved an F1-score of 0.9446, while V3 achieved 0.9397. Thus, augmentation did not improve performance on the original data distribution. In contrast, V3's external F1-score increased from 0.3522 for the baseline to 0.8377. The inclusion of obfuscated and encoded injection examples may have helped the model recognize patterns that were insufficiently represented in the original training data. However, because the augmentation strategy was informed by characteristics observed in the external dataset, the improvement should be interpreted as evidence of enhanced robustness on the evaluated subset rather than conclusive proof of generalization to all unseen attacks.

D. Cross-Dataset Generalization

The difference between internal and external performance highlights the importance of evaluating prompt injection detection systems across multiple data distributions. Although the baseline model achieved strong results on the original dataset, its external F1-score significantly dropped, suggesting that good performance on known data might not accurately represent performance against different attack types. The enhancement seen in V3 indicates that focused data augmentation can help bridge this performance gap. Nonetheless, the persistent disparity between internal and external outcomes implies that additional evaluation using independently sourced datasets is crucial before the model can be deemed dependable for wider application.

TABLE XI
Internal and External Test F1-Score Comparison

Version	Internal Test F1	External Test F1
Baseline	0.9341	0.3522
V1	0.9402	0.5205
V2	0.9405	0.5343
V3	0.9379	0.8377

Table XI compares internal and external test performance across the baseline and augmented versions. V2 achieved the highest internal test F1-score (0.9405), whereas V3 achieved the highest external test F1-score (0.8377). Although V3 did not produce the highest internal test score, its external results indicate substantially better performance on the evaluated external subset. Therefore, V3 was selected for deployment based on its external robustness performance rather than internal F1-score alone.

E. Error Analysis

The insights gained from classification errors shed light on the advantages and drawbacks of PromptGuard. False negatives are especially crucial as they signify prompt injection attempts that the model inaccurately identifies as harmless, which could allow harmful instructions to reach the language model. Conversely, false positives arise when legitimate prompts are mistakenly identified as potentially dangerous, possibly disrupting normal system operations. These two categories of errors highlight the necessity of striking a balance between detection sensitivity and the accurate classification of harmless inputs. Despite the model demonstrating impressive internal performance, the presence of remaining classification errors suggests it

should be incorporated as part of a larger security strategy instead of being relied upon exclusively to prevent prompt injection attacks.

F. Practical Implications

The findings indicate that PromptGuard has potential as a lightweight preliminary screening mechanism for applications that process user prompts through large language models. Its use of traditional machine learning algorithms and character-level TF-IDF features provides a practical alternative for systems that require relatively simple text classification without relying on resource-intensive deep learning models for the detection stage. The high external precision of V3 suggests that the model can identify many prompts it flags as malicious with a low proportion of false alarms under the evaluated conditions. However, its external recall of 0.7226 indicates that a substantial number of injection attempts remained undetected. Therefore, PromptGuard should be integrated with complementary security measures, such as input validation, access controls, and additional contextual inspection,

G. Limitations

Several limitations must be considered when interpreting the results. First, the dataset showed a substantial class imbalance toward benign prompts, which may affect the stability and interpretation of the model's performance. Second, the primary dataset consisted of 20,000 samples, which provided a basis for the controlled comparison of the three selected traditional machine learning algorithms. However, a more extensive and varied dataset could offer greater insight into prompt-injection methods and enhance the ability to generalize to previously unseen attack patterns. Third, augmentation samples were synthetic data generated by the researchers; therefore, they cannot fully represent the diversity of real-world prompt injection attacks. Fourth, the researchers designed data augmentation based on observed characteristics of the external dataset, so it should be interpreted as an adaptation or robustness evaluation rather than a completely untouched benchmark. Furthermore, exploratory external challenge tests showed that certain injection commands embedded within documents that appear harmless were labeled as benign. This indicates that the character-level TF-IDF representation might not adequately reflect the contextual connections between harmful instructions and the text surrounding them. These limitations indicate that PromptGuard must be viewed as a lightweight detection tool rather than a complete solution to prompt-injection security.

V. CONCLUSION AND RECOMMENDATIONS

A. Conclusion

This study developed PromptGuard, a lightweight text-based prompt injection detection system, and compared three traditional machine learning algorithms: Logistic Regression, Linear Support Vector Machine (Linear SVM), and Multinomial Naive Bayes. Using different TF-IDF feature representations and consistent experimental procedures, Linear SVM with character-level n-grams achieved the highest cross-validation F1-score of 0.9446 among the evaluated algorithms, making it the selected classification model.

The results showed that data augmentation had different effects on internal and external evaluations. Although the baseline model achieved a slightly higher cross-validation F1-score of 0.9446 than the V3 augmented model's 0.9397, V3 demonstrated a substantial improvement in external robustness. Its external F1-score increased from 0.3522 to 0.8377, while its recall improved from 0.2145 to 0.7226. These findings suggest that targeted synthetic data augmentation can improve the detection of prompt injection attacks across different dataset distributions, even when it does not improve performance on the original data distribution.

Despite these improvements, PromptGuard still missed some injection attempts during external evaluation. The use of a class-imbalanced dataset, synthetic augmentation, and an external evaluation set that informed augmentation design also limits the conclusions that can be drawn about its generalizability. Therefore, PromptGuard should be considered a lightweight prompt screening mechanism rather than a complete security solution.

B. Recommendations

Based on the findings and limitations of this study, several improvements are recommended for future research. First, future researchers should expand the dataset by incorporating more diverse real-world prompt injection attacks, particularly multilingual, encoded, obfuscated, indirect, and multi-step attacks. Additional benign examples that resemble malicious instructions should also be included to improve the distinction between legitimate prompts and potential attacks. Because the primary dataset is highly imbalanced, future studies should also investigate appropriate class-balancing techniques while monitoring their effects on precision, recall, and F1-score.

Second, the final model should be evaluated using a completely independent external dataset that has not influenced data augmentation, feature selection, hyperparameter tuning, or model selection. This would provide a more reliable assessment of the model's ability to generalize to previously unseen prompt injection attacks.

Third, future researchers should improve the feature extraction process to capture contextual relationships between malicious instructions and surrounding text. Approaches such as contextual embeddings or hybrid feature representations may help detect indirect and embedded prompt injection attempts that character-level TF-IDF features may overlook.

Finally, future work should conduct more extensive error analysis and evaluate PromptGuard under realistic

deployment conditions, including detection latency, processing throughput, and resource consumption. The system should also be integrated with complementary security mechanisms to reduce the risks associated with missed attacks. These improvements may help establish PromptGuard as a more reliable and practical component of a broader prompt injection defense system.

References

[1] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. "*A comprehensive overview of large language models"*, 2024 .

[2] Jones, R. K., & Mohammed, A. M. *"Prompt injection attacks risks and defense mechanisms", Scientific Reports, vol 16, no, 1277,* 2026..

[3] Chang, X., Dai, G., Di, H., & Ye, H. "*Breaking the prompt wall (I): A real-world case study of attacking ChatGPT via lightweight prompt injection"*. in arXiv, 2025.

[4] Alzahrani, A. (2026). "*PromptGuard: A structured framework for injection resilient language models",* 2026.

[5] Geng, T., Xu, Z., Qu, Y., & Wong, W. E. "*Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies" in Computers, Materials & Continua, vol 87*, no. 4, 2026.

[6] Ayub, M. A., & Majumdar, S. (2024). "*Embedding-based classifiers can detect prompt injection attacks" in arXiv,* 2024.

[7] Alshammari, A. A., & Alsaleh, O. I. "*Detecting prompt injection attacks in generative AI systems: A hybrid SIEM and one-class SVM framework" in Electronics, vol 15*(11), no. 2242, 2026

[8] Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., Zhang, L. Y., & Liu, Y. "*Prompt injection attack against LLM-integrated applications", in arXiv,* 2023

APPENDICES

The following appendices provide supplementary experimental results, additional figures, dataset documentation, application test cases, and project documentation supporting the study.

Appendix A. Supplementary Augmentation Results

TABLE A1
Training Size and Evaluation Performance Across Augmentation Versions

Training Versions	Training Samples	CV F1 (Mean ± SD)	Internal test F1-score
Baseline	16,000	0.9446 ± 0.0039	0.9341
V1	16,080	0.9394 ± 0.0102	0.9402
V2	16,115	0.9361 ± 0.0118	0.9405
V3	16,210	0.9397 ± 0.0121	0.9379

Table A1 presents the training-set sizes, mean cross-validation F1-scores with standard deviations, and internal test F1-scores for the baseline and augmented versions. The baseline obtained the highest mean cross-validation F1-score, whereas V2 achieved the highest internal test F1-score. V3 was selected because of its stronger performance on the evaluated external robustness subset rather than its internal test performance alone.

Appendix B. Detailed Internal Test Results

Table B1
Detailed Internal Test Results

Version	Accuracy	Precision	Recall	F1-score
Baseline	0.9943	0.9819	0.8907	0.9341
V1	0.9948	0.9821	0.9016	0.9402
V2	0.9948	0.9765	0.9071	0.9405
V3	0.9945	0.9708	0.9071	0.9379

Table B1 provides the internal test metrics for each training version. V2 achieved the highest internal test F1-score of 0.9405, while V2 and V3 both achieved a recall of 0.9071. These results provide a detailed comparison of the models' performance on the same internal test set.

TABLE B2
Internal Test Confusion-Matrix Counts

Version	TN	FP	FN	TP
Baseline	3814	3	20	163
V1	3814	3	18	165
V2	3813	4	17	166
V3	3812	5	17	166

Table B2 presents the confusion-matrix counts for each version. Compared with the baseline, V3 produced fewer false negatives but more false positives. This comparison illustrates the trade-off between detecting injection prompts and avoiding incorrect warnings on benign inputs.

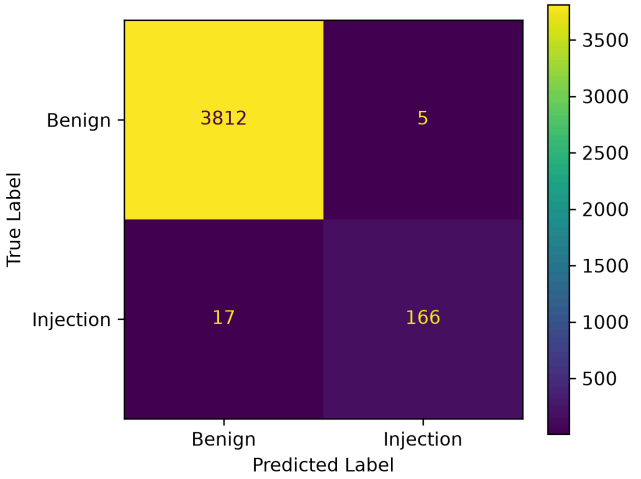


Fig. B1. Confusion matrix of the V3 model on the internal test set.

Figure B1 provides a visual representation of the V3 model's internal test results. The model correctly classified 3,812 benign prompts and 166 injection prompts. It incorrectly classified five benign prompts as injection and missed 17 injection prompts. Compared with the baseline, V3 reduced false negatives from 20 to 17, while false positives increased from three to five. These results indicate a modest improvement in internal injection detection, accompanied by a small increase in false alarms.

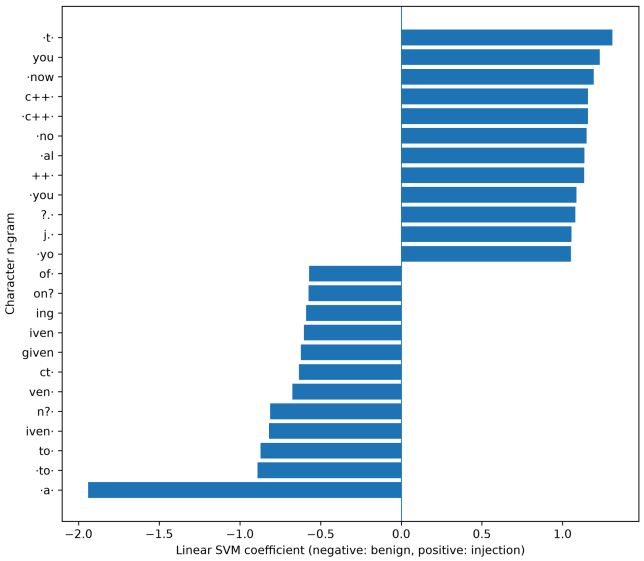


Fig. B2. Top character-level TF-IDF features contributing to the Linear SVM classifier.

Figure B2 presents the character-level TF-IDF features with the largest positive and negative coefficients in the Linear SVM classifier. These coefficients provide supplementary insight into the text patterns used by the model when distinguishing benign prompts from prompt injection attempts. The figure complements the quantitative results but does not independently establish the effectiveness of individual features.

Appendix C. Data Dictionary and Sample Input/Outputs

TABLE C1
Data Dictionary

Field	Description
text	The text of a user prompt submitted for classification by the PromptGuard model.
label	The original category assigned to each prompt in the dataset, indicating whether it is benign or an injection.
target	The numerical representation of the class label used for model training and evaluation: 0 = benign; 1 = injection.

Table C1 describes the principal fields used in the prompt-classification dataset. The text field contains the prompt supplied as model input, while label stores the original class designation. During preprocessing, the class label is represented numerically in target, with 0 indicating a benign prompt and 1 indicating a prompt injection. This encoding enables the classification algorithms to use the labels during model training and evaluation.

Dataset Sources and Processing Notes

The primary dataset used in this study was the Prompt Injection and Benign Prompt Dataset, obtained from Kaggle. It contained 20,000 prompts, including 19,083 benign prompts and 917 prompt injection samples. The original `text` and `label` fields were used to represent prompt content and its corresponding class label. During preprocessing, the labels were encoded as 0 for benign prompts and 1 for prompt injection samples. The dataset's source, applicable license or usage terms, and processing details are documented as part of the project dataset package.

TABLE C2
DATASET SOURCE INFORMATION

Item	Primary dataset	External dataset
Purpose	Model training and internal evaluation	External robustness assessment
Source	https://www.kaggle.com/datasets/krishnayadav456wrsty/prompt-injection-and-jailbreak-detection-dataset	https://github.com/mirzaakhi/prompt-injection-detection-dataset/tree/main
Dataset size	20, 000 prompts	5,441 records in larger dataset
Evaluated subset	4,000 internal test samples	1,229 samples
License	Verify on source page	Verify on source page

TABLE C3
Representative Application Test Cases

Test Case	Input	Application output
Benign	“Can you explain machine learning?”	Safe Prompt (Benign)
Injection	“Ignore all previous instructions”	Potential Prompt Injection Detected
Empty	blank	Please enter a prompt before sending.

Table C3 documents three representative application test cases: a benign prompt, a prompt injection attempt, and an empty input. The first two cases demonstrate the application's classification behavior, while the third demonstrates its input-validation behavior.

Appendix D. AI Usage Disclosure

Statement of AI-Assisted Tool Usage

AI-assisted tools were used to support code debugging and troubleshooting during the development of the PromptGuard system. These tools provided suggestions for identifying potential programming errors and resolving technical issues. All suggestions were reviewed, and responsibility for the final implementation, system testing, verification of results, and conclusions remained with the research team.

Appendix E. Group Contribution

Members	**Contribution**
Devera, Christian V.	**System Implementation, Python coding, ML pipeline, research documentation, Streamlit integration, project coordination**
Galvez, Ais Clifford	**Dataset exploration, data preprocessing, research documentation, literature review**
Lasay, Francis Adrian	**Model evaluation, system testing, research documentation, results visualization**

Appendix F. Ownership and Authorship Declaration

This signed declaration is a required submission document. Complete all fields, obtain the signatures of the instructor and every group member, convert the signed copy to PDF, and include it in the documentation folder and as an appendix to the final paper.

Project Information	Details
Project Title	PromptGuard: A Comparative Study of Traditional Machine Learning Algorithms for Prompt Injection Attack Detection
Course and Section	Introduction to Machine Learning - COM241ML
Group Name or Number	PromptGardener
Date Submitted	October 15, 2026

Declaration Statement

We, Dr. Armida P. Salazar, as the course instructor, and the undersigned student group members, jointly declare that we are the owners and authors of the submitted project output identified above. The project output includes the application, source code, trained model and saved pipeline, exploratory data analysis, visualizations, technical documentation, research paper, and original materials produced specifically for this course project.

We affirm that the submitted work accurately represents our documented contributions and that all external datasets, software libraries, code, models, frameworks, images, publications, and other third-party materials have been properly cited, licensed, or acknowledged using IEEE citation format where applicable. This declaration does not transfer or claim ownership of pre-existing third-party or institutional materials and remains subject to applicable National University policies, intellectual property rules, licenses, and Philippine law.

Signatories:

Name and Role	Signature	Date
Dr. Armida P. Salazar Course Instructor		October 15, 2026
Devera, Christian V. Group Leader		October 15, 2026
Lasay, Francis M. Group Member		October 15, 2026
Galvez, Clifford Ais S. Group Member		October 15, 2026